

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ЛЬВІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ІВАНА
ФРАНКА

Факультет прикладної математики та інформатики

Кафедра теорії оптимальних процесів

КУРСОВА РОБОТА
«ОСОБЛИВОСТІ РОЗРОБКИ ВЕБ-ДОДАТКУ У СЕРЕДОВИЩІ
NODE.JS ТА ANGULAR»

Виконав: студент групи **ПМА-31**
спеціальності **124 – системний аналіз**

(підпис)

Стець Т. В.

Керівник

(підпис)

Голуб Б. М.

Львів – 2026

Зміст

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ, СКОРОЧЕНЬ І ТЕРМІНІВ.....	2
ВСТУП.....	3
РОЗДІЛ 1 ТЕОРЕТИЧНІ ОСНОВИ РОЗРОБКИ СУЧАСНИХ ВЕБ-ДОДАТКІВ.....	5
1.1. Поняття веб-додатку та архітектура клієнт – сервер	5
1.2. Особливості технології SPA	6
1.3. Особливості платформи Node.js	6
1.4. Особливості фреймворку Angular	7
1.5. REST API як основа взаємодії між клієнтом та сервером.....	9
1.6. Технології MongoDB та JWT-автентифікації	9
РОЗДІЛ 2 АНАЛІЗ ТА ПРОЄКТУВАННЯ ВЕБ-ДОДАТКУ ДЛЯ КЕРУВАННЯ ЗАДАЧАМИ	11
2.1. Постановка задачі та функціональні вимоги	11
2.2. Обґрунтування вибору стеку Node.js + Angular	11
2.3. Порівняння Node.js + Angular з альтернативними технологіями	12
2.4. Архітектура системи.....	13
2.5. Реалізація системи авторизації та ролей	14
2.6. Проєктування бази даних	15
2.7. Проєктування інтерфейсу користувача	17
РОЗДІЛ 3 РЕАЛІЗАЦІЯ ТА ТЕСТУВАННЯ ВЕБ-ДОДАТКУ	20
3.1. Реалізація серверної частини.....	20
3.2. Реалізація клієнтської частини	22
3.3. Реалізація системи автентифікації та захисту даних.....	22
3.4. Реалізація аналітики та системи ролей	23
3.5. Тестування веб-додатку.....	23
3.6. Аналіз результатів роботи системи.	24
ВИСНОВКИ	25
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ	27

ПЕРЕЛІК УМОВНИХ ПОЗНАЧЕНЬ, СКОРОЧЕНЬ І ТЕРМІНІВ

SPA – Single Page Application, односторінковий веб-додаток.

API – Application Programming Interface, інтерфейс програмної взаємодії.

REST API – архітектурний стиль побудови веб-сервісів на основі HTTP.

JWT – JSON Web Token, механізм авторизації та передачі токенів доступу.

CRUD – Create, Read, Update, Delete — основні операції роботи з даними.

MongoDB – документоорієнтована NoSQL база даних.

Express.js – фреймворк для Node.js, призначений для створення серверних застосунків.

Angular – фронтенд-фреймворк для створення SPA-додатків.

Mongoose – ODM-бібліотека для роботи Node.js з MongoDB.

TypeScript – типізована надбудова над JavaScript.

UI – User Interface, користувацький інтерфейс.

UX – User Experience, користувацький досвід.

HTTP – HyperText Transfer Protocol.

JSON – формат обміну даними JavaScript Object Notation.

ВСТУП

Актуальність теми курсової роботи зумовлена активним розвитком веб-технологій та переходом від класичних багатосторінкових сайтів до сучасних інтерактивних веб-додатків. Сучасний користувач очікує швидкої роботи інтерфейсу, динамічного оновлення даних без повного перезавантаження сторінок, зручної навігації та можливості працювати із системою у режимі, наближеному до настільних програм.

Одним із найбільш поширених підходів до створення таких систем є використання архітектури SPA (Single Page Application). У SPA-додатках більшість логіки інтерфейсу виконується на стороні клієнта, а сервер забезпечує надання даних через API. Для реалізації таких систем доцільно використовувати сучасні інструменти фронтенд- та бекенд-розробки, зокрема Angular для клієнтської частини та Node.js для серверної.

Node.js є сучасним серверним середовищем виконання JavaScript, побудованим на рушії V8. Воно забезпечує високу продуктивність, підтримує неблокувальну модель введення-виведення та дозволяє використовувати єдину мову програмування як для серверної, так і для клієнтської частини проєкту. Angular, у свою чергу, є потужним фреймворком для побудови масштабованих SPA-додатків, який підтримує компонентний підхід, маршрутизацію, роботу з формами та інтеграцію з REST API.

У межах курсової роботи реалізується веб-додаток для керування задачами користувача. Система підтримує реєстрацію та авторизацію користувачів, створення персональних і спільних задач, розмежування доступу між ролями, аналітику виконання задач, систему налаштувань та адміністративну панель.

Метою курсової роботи є проєктування та реалізація сучасного веб-додатку для керування задачами з використанням Node.js, Angular та MongoDB, а також аналіз особливостей обраного стеку технологій.

Для досягнення поставленої мети необхідно виконати такі завдання:

1. дослідити теоретичні засади побудови веб-додатків та архітектури клієнт – сервер;

2. проаналізувати особливості платформ Node.js та Angular;
3. дослідити принципи роботи REST API та JWT-автентифікації;
4. порівняти стек Node.js + Angular з альтернативними технологіями;
5. спроектувати структуру веб-додатку для керування задачами;
6. реалізувати серверну частину, базу даних та клієнтський інтерфейс;
7. реалізувати систему ролей та захисту даних;
8. провести тестування системи та проаналізувати результати роботи.

Об'єктом дослідження є процес розробки сучасних веб-додатків.

Предметом дослідження є засоби та методи реалізації SPA-додатків з використанням Node.js та Angular.

Практичне значення роботи полягає у створенні прикладного веб-додатку для керування задачами, який може використовуватися як основа для подальшого розвитку корпоративних інформаційних систем.

РОЗДІЛ 1 ТЕОРЕТИЧНІ ОСНОВИ РОЗРОБКИ СУЧАСНИХ ВЕБ-ДОДАТКІВ

У сучасному світі веб-технології є основою більшості інформаційних систем. Практично кожна сфера діяльності людини використовує веб-додатки: банківські системи, електронну комерцію, системи керування персоналом, CRM-платформи, системи аналітики та корпоративні портали. Розвиток інтернет-технологій призвів до переходу від простих статичних сторінок до складних інтерактивних SPA-додатків, які працюють у режимі реального часу та забезпечують високий рівень взаємодії з користувачем.

Сучасний веб-додаток повинен відповідати великій кількості вимог. Серед основних вимог можна виділити швидкість роботи, масштабованість, безпечність, адаптивність інтерфейсу, підтримку різних типів пристроїв та можливість одночасної роботи великої кількості користувачів. Саме тому під час створення сучасних веб-систем використовуються спеціалізовані фреймворки та серверні платформи.

Одними з найбільш популярних технологій для побудови SPA-додатків є Node.js та Angular. Angular забезпечує створення структурованого клієнтського інтерфейсу, а Node.js дозволяє створювати продуктивні серверні застосунки та REST API. Використання цих технологій у межах одного проєкту дозволяє отримати єдину екосистему JavaScript/TypeScript, що значно спрощує розробку та підтримку системи.

1.1. Поняття веб-додатку та архітектура клієнт – сервер

Веб-додаток — це програмна система, доступ до якої користувач отримує через браузер або інший веб-клієнт. На відміну від звичайних статичних веб-сторінок, веб-додаток підтримує обробку даних, взаємодію з базами даних, автентифікацію користувачів, а також динамічне оновлення інтерфейсу.

Основою більшості сучасних веб-систем є модель клієнт – сервер. Клієнтська частина відповідає за відображення інтерфейсу та взаємодію з користувачем, а серверна — за бізнес-логіку, обробку запитів та взаємодію з базою даних.

Така архітектура забезпечує чіткий розподіл відповідальності між компонентами системи. Клієнт надсилає HTTP-запити до сервера, сервер виконує обробку та повертає відповідь у форматі JSON.

У традиційних багатосторінкових сайтах кожна дія користувача призводить до повного перезавантаження сторінки. У сучасних SPA-додатках після першого завантаження більшість змін виконується без перезавантаження, що значно підвищує швидкість роботи інтерфейсу.

1.2. Особливості технології SPA

SPA (Single Page Application) — це архітектурний підхід до створення веб-додатків, при якому всі основні елементи інтерфейсу завантажуються один раз, а подальша взаємодія відбувається динамічно через API.

Основними перевагами SPA є:

- швидка робота інтерфейсу;
- відсутність повного перезавантаження сторінки;
- кращий користувацький досвід;
- можливість створення складних інтерактивних систем;
- розділення фронтенда і бекенда.

До недоліків SPA можна віднести складнішу початкову конфігурацію проєкту, вищі вимоги до структури коду та потребу у реалізації маршрутизації й системи безпеки.

Для системи керування задачами архітектура SPA є оптимальною, оскільки користувач часто виконує однотипні дії: створює задачі, змінює їхній статус, переглядає аналітику та редагує інформацію.

1.3. Особливості платформи Node.js

Node.js набув широкого поширення у сфері веб-розробки завдяки своїй продуктивності та простоті інтеграції з сучасними JavaScript-фреймворками. Платформа побудована на рушії Google V8, який компілює JavaScript у машинний код. Це дозволяє забезпечити високу швидкість виконання серверних застосунків.

Однією з головних переваг Node.js є асинхронна модель роботи. У традиційних багатопотокових серверних технологіях кожен новий запит може

створювати окремий потік виконання, що призводить до значного навантаження на систему. У Node.js використовується Event Loop, який дозволяє ефективно обробляти велику кількість одночасних підключень без створення великої кількості потоків.

Важливою складовою екосистеми Node.js є npm — менеджер пакетів, який містить тисячі готових бібліотек. Завдяки npm розробник може швидко інтегрувати у проєкт:

- систему авторизації;
- механізми роботи з базами даних;
- бібліотеки для тестування;
- middleware-компоненти;
- інструменти логування;
- бібліотеки для роботи з файлами.

У межах курсової роботи використовувались такі бібліотеки:

- express — для реалізації сервера;
- mongoose — для роботи з MongoDB;
- jsonwebtoken — для JWT-автентифікації;
- bcryptjs — для хешування паролів;
- cors — для підтримки міждоменних запитів.

Node.js добре підходить для створення:

- REST API;
- чатів у реальному часі;
- SPA-систем;
- панелей адміністрування;
- систем аналітики.

Саме тому Node.js було обрано як основу серверної частини веб-додатку.

1.4. Особливості фреймворку Angular

Angular є одним із найпотужніших сучасних фреймворків для створення SPA-додатків. Він розроблений компанією Google та підтримує компонентну архітектуру, що дозволяє розділяти інтерфейс на незалежні функціональні частини.

Кожен компонент Angular складається з:

- TypeScript-класу;
- HTML-шаблону;
- CSS-стилів.

Такий підхід забезпечує високу структурованість коду та дозволяє повторно використовувати компоненти у різних частинах системи.

Angular використовує TypeScript — типізовану надбудову над JavaScript. Завдяки цьому зменшується кількість помилок у великих проєктах, а також спрощується підтримка коду.

У межах створеного веб-додатку Angular використовується для:

- реалізації системи маршрутизації;
- створення форм входу та реєстрації;
- відображення списку задач;
- реалізації аналітики;
- реалізації адміністративної панелі;
- роботи з HTTP API.

Особливу роль у Angular відіграють сервіси. Сервіси дозволяють винести логіку взаємодії з сервером у окремі класи, що забезпечує кращу структуру проєкту.

У проєкті було реалізовано:

- AuthService — сервіс авторизації;
- TaskService — сервіс роботи із задачами;
- AnalyticsService — сервіс аналітики.

Angular також підтримує guards — спеціальні механізми захисту маршрутів.

У системі використовуються:

- authGuard — перевірка авторизації;
- adminGuard — перевірка ролі адміністратора.

Завдяки цьому неавторизований користувач не може отримати доступ до захищених сторінок системи.

1.5. REST API як основа взаємодії між клієнтом та сервером

REST API є одним із найбільш поширених підходів до побудови сучасних веб-сервісів. Основою REST є використання HTTP-запитів для виконання операцій над ресурсами. Такий підхід дозволяє забезпечити просту та зрозумілу взаємодію між клієнтською та серверною частиною веб-додатку.

У REST API кожен ресурс має власний URL-адрес, а взаємодія із сервером здійснюється через стандартні HTTP-методи.

Основні HTTP-методи:

- GET — отримання даних;
- POST — створення нових записів;
- PUT — оновлення інформації;
- DELETE — видалення даних.

У межах розробленого веб-додатку REST API використовується для взаємодії Angular-клієнта із сервером Node.js. Клієнтська частина надсилає HTTP-запити до сервера для отримання інформації про задачі, створення нових задач, оновлення статусу виконання та видалення даних.

Для роботи із REST API в Angular використовується модуль HttpClient, який забезпечує асинхронний обмін даними між клієнтом та сервером.

Серверна частина реалізована за допомогою Express.js. Для кожного типу операцій створено окремі маршрути API, наприклад:

- /api/tasks — робота із задачами;
- /auth/login — авторизація користувача;
- /auth/register — реєстрація нового користувача.

Передача даних між клієнтом та сервером виконується у форматі JSON, що забезпечує простоту інтеграції та високу швидкість обробки інформації.

1.6. Технології MongoDB та JWT-автентифікації

MongoDB є сучасною документоорієнтованою NoSQL базою даних. На відміну від реляційних СУБД, MongoDB зберігає інформацію у вигляді BSON-документів, які є розширенням формату JSON.

Використання MongoDB дозволяє забезпечити гнучке зберігання даних та спрощує роботу із динамічними структурами документів. Це особливо важливо для веб-додатків, у яких структура даних може змінюватися під час розвитку системи.

Основними перевагами MongoDB є:

- гнучка структура даних;
- висока швидкість роботи;
- простота масштабування;
- зручна інтеграція з Node.js;
- підтримка роботи з JSON-подібними документами.

У межах проєкту MongoDB використовується для зберігання інформації про користувачів та задачі. Для взаємодії із базою даних використовується бібліотека Mongoose, яка дозволяє створювати моделі даних та виконувати CRUD-операції.

Для забезпечення безпеки у веб-додатку реалізовано JWT-автентифікацію. Після успішного входу користувача сервер генерує JWT-токен, який надалі використовується для доступу до захищених маршрутів API.

Токен передається у заголовках HTTP-запитів та перевіряється сервером за допомогою middleware-компонента authMiddleware. У разі відсутності або недійсності токена доступ до API блокується.

Використання JWT дозволяє:

- забезпечити захист даних користувачів;
- контролювати доступ до функцій системи;
- реалізувати систему ролей;
- підтримувати авторизовану роботу користувача без повторного введення пароля.

РОЗДІЛ 2 АНАЛІЗ ТА ПРОЄКТУВАННЯ ВЕБ-ДОДАТКУ ДЛЯ КЕРУВАННЯ ЗАДАЧАМИ

2.1. Постановка задачі та функціональні вимоги

У межах курсової роботи необхідно реалізувати веб-додаток для керування задачами.

Основні функції системи:

- реєстрація користувача;
- авторизація;
- створення задач;
- редагування задач;
- видалення задач;
- підтримка персональних та спільних задач;
- перегляд аналітики;
- підтримка ролей користувачів;
- адміністративна панель.

Система повинна забезпечувати швидку взаємодію між клієнтською та серверною частиною, підтримувати безпечну авторизацію користувачів та зручний інтерфейс для роботи із задачами.

Окрему увагу приділено підтримці командної роботи. Користувачі можуть створювати shared-задачі та надавати доступ іншим користувачам системи.

Також у веб-додатку реалізовано підтримку мультимовного інтерфейсу та темного режиму оформлення, що покращує зручність використання системи.

2.2. Обґрунтування вибору стеку Node.js + Angular

Використання Node.js та Angular дозволяє створити сучасний SPA-додаток із єдиною екосистемою JavaScript/TypeScript.

Angular забезпечує побудову складного інтерфейсу, використовує компонентний підхід, який дозволяє розділяти функціональність системи на окремі незалежні модулі. Це спрощує підтримку та масштабування веб-додатку. Node.js дозволяє ефективно працювати із великою кількістю одночасних HTTP-запитів завдяки асинхронній моделі виконання. Використання JavaScript та TypeScript як на

клієнтській, так і на серверній частині спрощує процес розробки та підтримки проєкту, а Node.js дозволяє швидко реалізувати REST API та інтеграцію з MongoDB.

2.3. Порівняння Node.js + Angular з альтернативними технологіями

Проведене порівняння показує, що використання стеку Node.js + Angular є оптимальним рішенням для створення SPA-додатку з підтримкою REST API, системи ролей та динамічного інтерфейсу користувача.

Даний стек забезпечує високу швидкість розробки, зручну інтеграцію між клієнтською та серверною частиною, а також хорошу масштабованість системи.

Таблиця 2.1 – Порівняння Node.js + Angular з альтернативними технологіями

Критерій	Node.js + Angular	PHP	Django	Spring
Швидкість розробки	Висока	Висока	Середня	Середня
SPA-підхід	Дуже зручний	Середній	Середній	Високий
Єдина мова	Так	Ні	Ні	Ні
Масштабованість	Висока	Середня	Висока	Дуже висока
Поріг входження	Середній	Низький	Середній	Високий

На основі порівняння можна зробити висновок, що стек Node.js + Angular є доцільним для реалізації даної системи, оскільки забезпечує зручну побудову SPA-інтерфейсу, швидку взаємодію з REST API та використання єдиної мови програмування у більшості частин проєкту. На відміну від PHP, такий підхід краще підходить для динамічних односторінкових веб-додатків. Порівняно з Django та Spring, стек Node.js + Angular має нижчий поріг входження та швидше дозволяє реалізувати клієнт-серверну взаємодію.

2.4. Архітектура системи

Архітектура створеного веб-додатку побудована за принципом розділення клієнтської та серверної частини. Такий підхід дозволяє забезпечити масштабованість, простоту підтримки та можливість подальшого розвитку системи.

Клієнтська частина реалізована за допомогою Angular та відповідає за:

- відображення інтерфейсу;
- роботу з формами;
- маршрутизацію;
- відправлення HTTP-запитів;
- відображення аналітики;
- роботу системи ролей.

Серверна частина реалізована за допомогою Node.js та Express і забезпечує:

- обробку HTTP-запитів;
- JWT-автентифікацію;
- взаємодію з MongoDB;
- перевірку прав доступу;
- бізнес-логіку системи.

MongoDB використовується як основне сховище даних. База даних містить інформацію про:

- користувачів;
- задачі;
- ролі;
- спільний доступ до задач.

У процесі проектування системи використовувався компонентний підхід. Це дозволило розділити функціональність між окремими модулями системи.

Структура серверної частини:

- models — моделі бази даних;
- routes — маршрути API;
- middleware — middleware-компоненти;

- config — конфігураційні файли.

Структура Angular-проєкту:

- components — компоненти інтерфейсу;
- services — сервіси роботи з API;
- guards — механізми захисту маршрутів;
- models — моделі даних;
- pages — сторінки застосунку.

Така архітектура дозволяє легко масштабувати систему та додавати новий функціонал.

Архітектурно система складається з трьох основних рівнів:

1. Клієнтська частина Angular.
2. Сервер Node.js + Express.
3. База даних MongoDB.

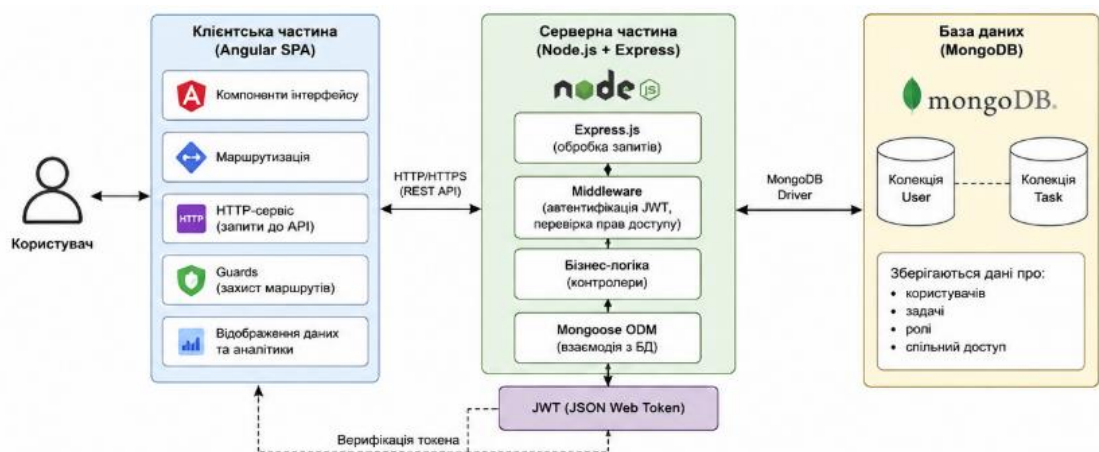


Рисунок 2.1 – Загальна архітектура веб-додатку

2.5. Реалізація системи авторизації та ролей

У веб-додатку реалізовано систему авторизації користувачів на основі JWT (JSON Web Token). Такий підхід дозволяє забезпечити захищений доступ до функціоналу системи та перевірку прав користувачів під час виконання HTTP-запитів.

Після успішної авторизації користувача серверна частина, реалізована за допомогою Node.js та Express, генерує JWT-токен. Згенерований токен зберігається

у localStorage браузера та автоматично передається у заголовках HTTP-запитів для подальшої взаємодії із сервером.

Для перевірки токена використовується middleware-компонент authMiddleware. Даний middleware перевіряє наявність токена, його коректність та дійсність. Якщо токен є недійсним або відсутнім, сервер повертає помилку авторизації та блокує доступ до API.

У системі реалізовано дві основні ролі користувачів:

- user — звичайний користувач;
- admin — адміністратор системи.

Користувач із роллю user має доступ лише до власних задач та задач, до яких йому надано спільний доступ через механізм sharedWith.

Адміністратор системи має розширені права доступу та може переглядати всі задачі незалежно від їхнього власника або типу задачі.

Для підтримки командної роботи реалізовано два типи задач:

- personal — особисті задачі;
- shared — командні задачі.

Командні задачі можуть бути доступні декільком користувачам одночасно. Для цього використовується поле sharedWith, у якому зберігається список користувачів зі спільним доступом до задачі.

На клієнтській частині Angular реалізовано механізм захисту маршрутів за допомогою authGuard та adminGuard. Доступ до сторінок системи перевіряється автоматично залежно від ролі користувача.

Система авторизації забезпечує:

- захист API від неавторизованого доступу;
- контроль прав доступу;
- підтримку ролей користувачів;
- безпечну роботу із персональними та командними задачами.

2.6. Проєктування бази даних

У веб-додатку використовується база даних MongoDB з назвою kursova_db. У ній створено дві основні колекції:

- users;
- tasks.

Колекція users зберігає інформацію про зареєстрованих користувачів системи. Документ користувача містить такі поля:

- `_id` — унікальний ідентифікатор документа;
- `username` — логін користувача;
- `password` — пароль користувача;
- `role` — роль користувача в системі;
- `team` — команда, до якої належить користувач.

Поле `role` використовується для розмежування прав доступу. Звичайний користувач має роль `user`, а адміністратор — `admin`. Адміністратор має право переглядати всі задачі системи незалежно від їхнього типу та власника.

Колекція tasks зберігає інформацію про задачі користувачів. Документ задачі містить такі поля:

- `_id` — унікальний ідентифікатор задачі;
- `title` — назва задачі;
- `completed` — статус виконання задачі;
- `category` — категорія задачі;
- `priority` — пріоритет задачі;
- `deadline` — кінцевий термін виконання;
- `taskType` — тип задачі: `personal` або `shared`;
- `owner` — логін користувача, який створив задачу;
- `sharedWith` — список користувачів, яким надано доступ до командної задачі;
- `steps` — список кроків виконання задачі;
- `notes` — додаткові нотатки до задачі;
- `createdAt` — дата створення задачі.

Зв'язок між колекціями `users` і `tasks` реалізовано через поле `owner`. Значення `owner` у задачі відповідає `username` користувача, який створив цю задачу.

Для командних задач використовується поле `sharedWith`. У ньому зберігається список логінів користувачів, які мають доступ до задачі. Якщо задача є особистою, поле `sharedWith` залишається порожнім.

Таким чином, один користувач може створити багато задач, а одна командна задача може бути доступна кільком користувачам.

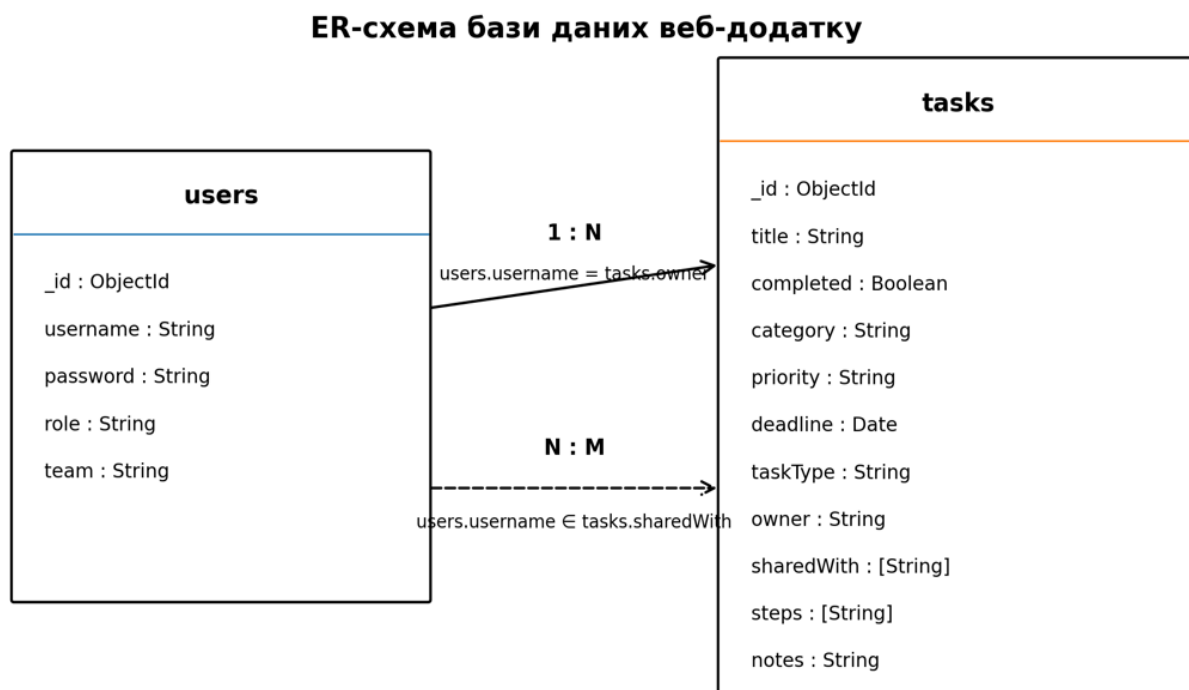


Рисунок 2.2 – ER-схема бази даних веб-додатку

2.7. Проектування інтерфейсу користувача

Інтерфейс системи реалізовано у сучасному стилі з використанням компонентного підходу Angular.

Основними сторінками системи є:

- сторінка входу;
- сторінка реєстрації;
- сторінка задач;
- адміністративна панель;
- сторінка аналітики;
- сторінка налаштувань користувача.

Для навігації використовується бічне меню із системою іконок.

Інтерфейс побудовано у вигляді SPA-додатку без перезавантаження сторінок. Для покращення користувацького досвіду реалізовано підтримку темного режиму та мультимовного інтерфейсу.

Сторінка задач дозволяє створювати, редагувати та переглядати задачі, а також працювати із командними задачами та системою shared-доступу.

Сторінка аналітики відображає статистику виконання задач за допомогою графіків та діаграм.

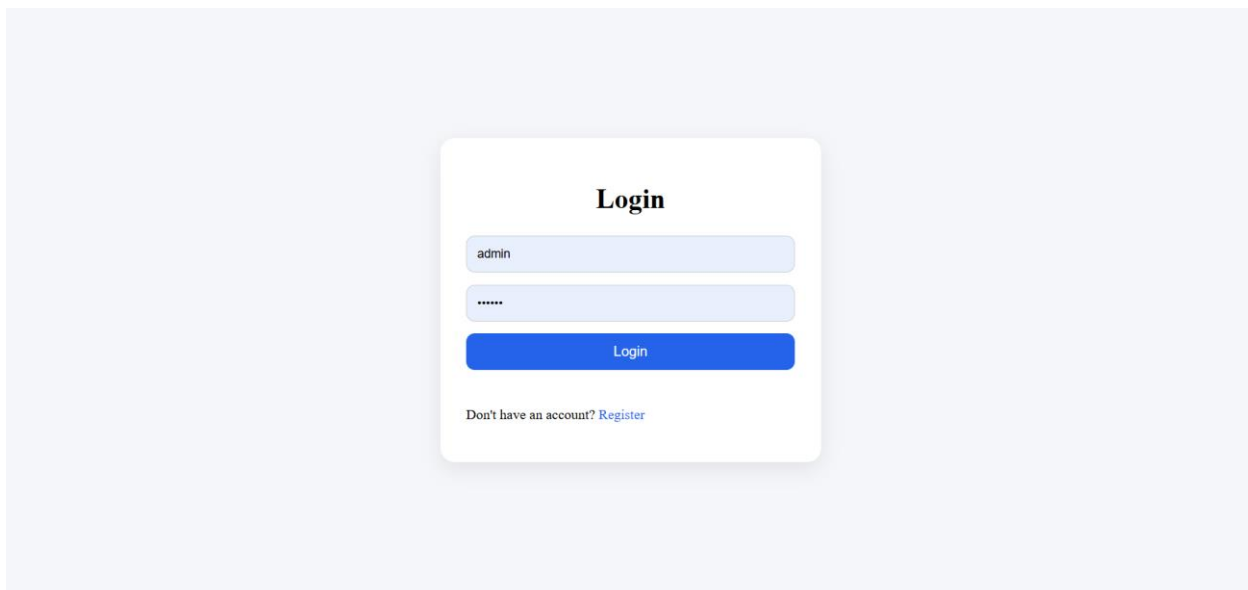


Рисунок 2.4 – Сторінка авторизації користувача

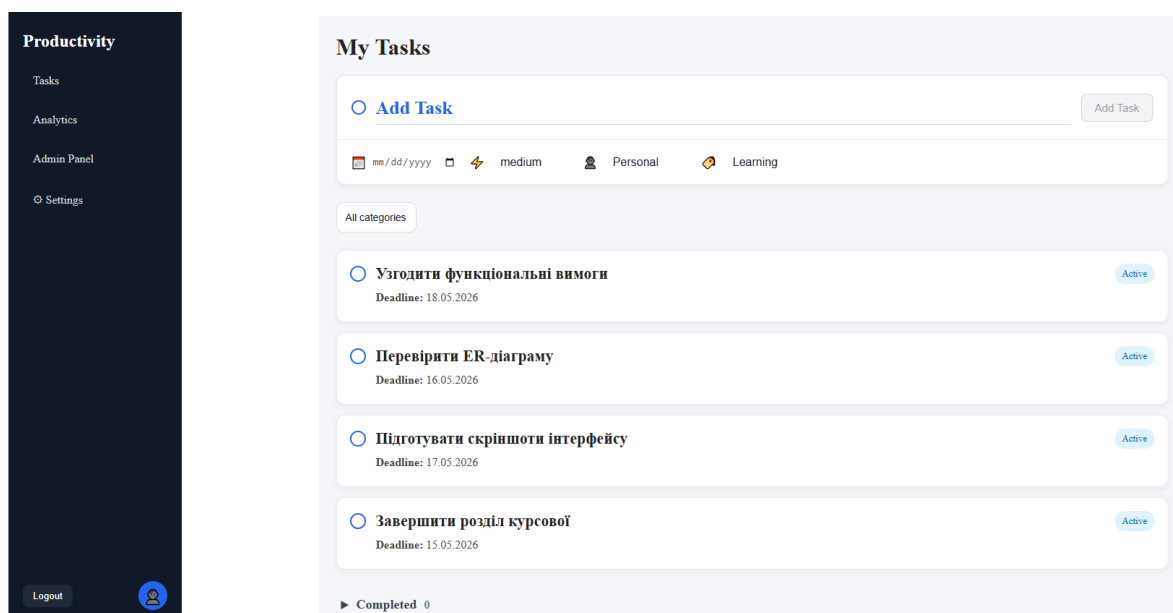


Рисунок 2.5 – Інтерфейс сторінки задач веб-додатку



Рисунок 2.6 – Сторінка аналітики виконання задач

РОЗДІЛ 3 РЕАЛІЗАЦІЯ ТА ТЕСТУВАННЯ ВЕБ-ДОДАТКУ

3.1. Реалізація серверної частини

Серверна частина системи реалізована за допомогою Node.js та Express. Основною задачею сервера є обробка HTTP-запитів, взаємодія з базою даних, перевірка JWT-токенів та реалізація бізнес-логіки системи.

Під час запуску сервера виконується:

1. Ініціалізація Express-застосунку.
2. Підключення middleware.
3. Підключення MongoDB.
4. Реєстрація маршрутів.
5. Запуск сервера.

Для підтримки обміну даними між Angular та Node.js використовувався middleware cors.

```
app.use(cors());
```

```
app.use(express.json());
```

Після цього відбувається підключення до MongoDB:

```
mongoose.connect('mongodb://127.0.0.1:27017/kursova_db');
```

У системі реалізовані такі API-маршрути:

- /auth/register — реєстрація нового користувача;
- /auth/login — авторизація користувача;
- GET /api/tasks — отримання списку задач;
- POST /api/tasks — створення нової задачі;
- PUT /api/tasks/:id — оновлення інформації про задачу;
- DELETE /api/tasks/:id — видалення задачі.

Для опису структури даних використовується Mongoose.

Модель задачі:

```
const taskSchema = new mongoose.Schema({  
  title: String,  
  completed: Boolean,
```

```
category: String,  
priority: String,  
deadline: Date,  
taskType: String,  
owner: String,  
sharedWith: [String],  
createdAt: Date,  
steps: [String],  
notes: String  
});
```

У системі реалізовано CRUD-функціонал:

- створення задач;
- перегляд задач;
- редагування задач;
- видалення задач.

Також реалізована підтримка спільних задач через поле `sharedWith`.

Сервер перевіряє роль користувача та забезпечує розмежування доступу до інформації.

Адміністратор має можливість переглядати всі задачі системи, а звичайний користувач — лише власні та спільні задачі.

Серверна частина реалізована за допомогою Node.js та Express.

Основний файл сервера:

```
const express = require('express');  
const mongoose = require('mongoose');  
const cors = require('cors');  
const app = express();  
app.use(cors());  
app.use(express.json());  
mongoose.connect('mongodb://127.0.0.1:27017/kursova_db');  
app.listen(3000, () => {
```

```
console.log('Server started');  
});
```

У наведеному коді створюється сервер Express, підключається MongoDB та запускається серверний застосунок.

3.2. Реалізація клієнтської частини

Клієнтська частина створена за допомогою Angular.

Основними компонентами системи є:

- Login - сторінка авторизації користувача;
- Register - сторінка реєстрації нового користувача;
- Tasks - основна сторінка для роботи із задачами;
- Analytics - сторінка аналітики виконання задач;
- Admin - адміністративна панель;
- Settings - сторінка налаштувань системи.

Для маршрутизації використовується Angular Router.

```
export const routes: Routes = [  
  { path: 'login', component: LoginComponent },  
  { path: 'tasks', component: TasksComponent },  
  { path: 'analytics', component: AnalyticsComponent }  
];
```

3.3. Реалізація системи автентифікації та захисту даних

У системі реалізована JWT-автентифікація.

Після успішного входу сервер генерує токен доступу.

```
module.exports = function(req, res, next) {  
  const token = req.header('Authorization');  
  if (!token) {  
    return res.status(401).json({ message: 'Access denied' });  
  }  
  try {  
    const decoded = jwt.verify(token, 'secret_key');  
    req.user = decoded;  
    next();  
  } catch (error) {
```

```
return res.status(401).json({ message: 'Invalid token' });
}
};
```

3.4. Реалізація аналітики та системи ролей

У веб-додатку реалізована система ролей:

- user;
- admin.

Звичайний користувач може переглядати лише власні задачі та спільні задачі.

Адміністратор має доступ до всіх задач системи.

Для реалізації аналітики використовувалась бібліотека ng2-charts.

Аналітика відображає:

- кількість виконаних задач;
- кількість активних задач;
- розподіл задач за категоріями;
- статистику виконання.

3.5. Тестування веб-додатку

Під час тестування було перевірено основні функціональні можливості системи.

Таблиця 3.1 – Результати тестування веб-додатку

№	Тестовий сценарій	Очікуваний результат	Результат
1	Вхід з правильними даними	Користувач переходить на сторінку задач	Успішно
2	Вхід з неправильним паролем	Система показує помилку авторизації	Успішно
3	Створення задачі	Нова задача додається до списку	Успішно
4	Редагування задачі	Зміни зберігаються у MongoDB	Успішно
5	Видалення задачі	Задача видаляється зі списку	Успішно
6	Доступ user до чужих задач	Чужі задачі не відображаються	Успішно
7	Доступ admin до всіх задач	Адміністратор бачить усі задачі	Успішно

№ Тестовий сценарій	Очікуваний результат	Результат
8 Запит до API без токена	Сервер повертає помилку доступу	Успішно
9 Відображення аналітики	Графіки будуються за даними задач	Успішно

3.6. Аналіз результатів роботи системи.

Результати тестування показали, що розроблена система успішно виконує всі поставлені функції та забезпечує стабільну роботу веб-додатку. Angular забезпечує швидку роботу інтерфейсу без перезавантаження сторінок та дозволяє ефективно організувати взаємодію між компонентами системи. Використання SPA-підходу позитивно вплинуло на швидкість роботи інтерфейсу та зручність користування системою. Node.js ефективно обробляє HTTP-запити та забезпечує швидку взаємодію із базою даних MongoDB. Використання асинхронної моделі виконання дозволяє підтримувати стабільну роботу серверної частини навіть при одночасному виконанні декількох запитів. Реалізована система ролей та JWT-автентифікації забезпечує безпечний доступ до даних та коректне розмежування прав користувачів. Це дозволяє обмежувати доступ до функціоналу системи відповідно до ролі користувача. Проведений аналіз підтвердив доцільність використання стеку Node.js + Angular + MongoDB для створення сучасних SPA-веб-додатків із підтримкою REST API, аналітики, системи ролей та механізмів захисту даних. Отримані результати демонструють можливість подальшого розвитку системи та розширення її функціональних можливостей.

ВИСНОВКИ

У процесі виконання курсової роботи було проведено комплексне дослідження сучасних технологій розробки SPA-веб-додатків. Було проаналізовано принципи роботи архітектури клієнт – сервер, REST API, технології JWT-автентифікації, особливості використання MongoDB та компонентний підхід Angular.

У ході виконання практичної частини було реалізовано повноцінний веб-додаток для керування задачами користувача. Система підтримує створення персональних та спільних задач, систему ролей, адміністративну панель, аналітику та механізми захисту даних.

Під час реалізації проєкту було використано сучасний стек технологій Node.js + Angular + MongoDB. Даний стек дозволив створити SPA-додаток із швидкою роботою інтерфейсу, розділенням клієнтської та серверної логіки, а також гнучкою структурою системи.

У процесі роботи було підтверджено, що Angular забезпечує ефективну реалізацію складного користувацького інтерфейсу, підтримує компонентний підхід та дозволяє створювати масштабовані SPA-системи.

Node.js продемонстрував високу продуктивність при обробці HTTP-запитів та взаємодії з MongoDB. Використання асинхронної моделі дозволяє ефективно працювати з великою кількістю одночасних запитів.

Важливою частиною системи стала реалізація JWT-автентифікації та middleware-захисту маршрутів. Це дозволило забезпечити безпечний доступ до інформації та реалізувати розмежування прав користувачів.

Також було реалізовано систему аналітики, яка дозволяє користувачу переглядати статистику виконання задач, аналізувати кількість виконаних та активних задач, а також отримувати інформацію про розподіл задач за категоріями.

Результати тестування підтвердили стабільність роботи системи та коректність реалізованого функціоналу. Усі основні функції працюють відповідно до поставлених вимог.

Практичним результатом курсової роботи є створений веб-додаток для керування задачами, який може бути основою для подальшого розвитку інформаційних систем такого типу.

У перспективі система може бути доповнена:

- push-сповіщеннями;
- інтеграцією з хмарними сервісами;
- підтримкою мобільних пристроїв;
- системою командної роботи;
- календарем та нагадуваннями;
- розширеною аналітикою.

Таким чином, поставлена мета курсової роботи була повністю досягнута, а всі поставлені завдання — успішно виконані.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Node.js Documentation. – Режим доступу: <https://nodejs.org/>
2. Angular Documentation. – Режим доступу: <https://angular.dev/>
3. MongoDB Documentation. – Режим доступу: <https://www.mongodb.com/docs/>
4. Express.js Documentation. – Режим доступу: <https://expressjs.com/>
5. JWT Introduction. – Режим доступу: <https://jwt.io/>
6. TypeScript Documentation. – Режим доступу: <https://www.typescriptlang.org/>
7. MDN Web Docs. – Режим доступу: <https://developer.mozilla.org/>
8. REST API Tutorial. – Режим доступу: <https://restfulapi.net/>
9. Chart.js Documentation. – Режим доступу: <https://www.chartjs.org/>
10. Mongoose Documentation. – Режим доступу: <https://mongoosejs.com/>
11. Freeman A. Pro Angular. — Apress, 2022.
12. Cantelon M. Node.js in Action. — Manning Publications, 2017.
13. Richardson L. RESTful Web APIs. — O'Reilly Media, 2013.